Developing and Distributing Applications with Docker

COMP3028 – Software Construction | Lecture 11

Node.js hands-on: LocalLoop Food Rescue API

Build → Run → Test → Share

No previous Docker experience required. Uses plain Node.js and built-in testing tools.

Spring 2026 | 16:9 | Beginner-friendly workshop

Today: the big picture

Lecture ideas (approx. 35 min)

1. The deployment problem
2. Images vs. containers
3. Dockerfiles, networking, volumes
4. Registries and distribution

Guided lab (approx. 75 min)

1. Run and test Node.js
2. Build and run an image
3. Debug and use Compose
4. Tag and share an image

Checkpoint

By the end: run the same LocalLoop application locally and in Docker, explain each step, and prepare a shareable version.

Learning outcomes

By the end of Lecture 11 you should be able to:

1. Explain **image**, **container**, **Dockerfile**, **registry** and **Compose**.
2. Package a Node.js HTTP API into a reproducible Docker image.
3. Run, inspect, test, stop and troubleshoot containers.
4. Configure published ports and environment variables safely.
5. Tag, push and pull a versioned image (or export/import offline).
6. Identify production limitations of the classroom prototype.

Links to COMP3028 practices: web services, testing, secure deployment and code reviews.

The real-world problem

Developer laptop

- "It works on my machine."
- Different Node.js versions
- Missing libraries and configuration
- Accidental dependence on local files

Tutor / deployment host

- Different operating system
- Different installed packages
- Port is already occupied
- Application will not start

Question: How can we package the runtime and application so someone else can start it consistently?

Case study: LocalLoop

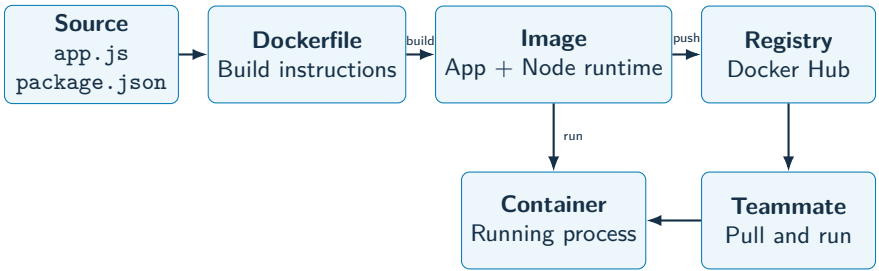
Official Spring 2026 scenario

A council commissions an online food-rescue platform to connect surplus-food businesses with community organisations, volunteers and residents.

- Publish surplus-food listings with pickup information.
- Discover and reserve available food.
- Manage user permissions, safety information and notifications.
- Track food-rescue impact.

Today's slice: a tiny teaching API for *listing, reservation and health checks*—not the full seven-function system. Source: COMP3028-SPR26-Case-Study.pdf (provided with the unit).

From source code to running service



Key idea: Docker builds a portable *image*; running an image starts a *container*.

Image, container and Dockerfile

Term	Meaning	Everyday analogy
Dockerfile	Recipe to assemble software layers	Cooking instructions
Image	Immutable packaged application + runtime	Prepared meal / template
Container	Running instance of an image	Meal being served / running copy
Registry	Service that stores and distributes images	Shared catalogue
Compose	Defines how services run together	Operating plan

Checkpoint

You can create several separate containers from one image. Their in-memory data is not automatically shared.

Containers vs. virtual machines

Virtual machine

- Guest operating system
- Separate kernel abstraction
- Often heavier startup/storage

Container

- Isolated application processes
- Shares host kernel through the container platform
- Often lighter and faster to launch

Docker Desktop typically runs Linux containers inside a lightweight Linux VM on Windows/macOS.
Containers are not a complete security boundary or a replacement for application security.

What Docker does / does not do

Docker helps with

- Packaging runtime and dependencies
- Environment consistency
- Repeatable startup
- Image versioning and sharing

Docker does not automatically

- Fix faulty application code
- Make an unsafe API secure
- Persist data without storage design
- Guarantee performance or availability

Checkpoint

Containerisation improves delivery; engineering quality still depends on code, tests and operations.

Lab setup: what you need

- **Docker Desktop** (Windows/macOS) or Docker Engine + Compose plugin (Linux); start the engine.
- **Node.js 22 or later** for running tests locally (lab image uses Node 24 LTS).
- A terminal: PowerShell, macOS Terminal or a Linux shell.
- The supplied `localloop-api` lab folder (no npm packages to download).
- Internet for the *first* image pull and optional Docker Hub sharing.

Node.js 24 remains supported as Maintenance LTS as of Nov. 2026; Node.js 26 is Active LTS. Source: nodejs.org release schedule.

Lab 0: verify your tools

```
docker --version
docker info
docker compose version
node --version
npm --version
```

Expected: Docker version displayed; `docker info` prints server details; `docker compose version` works.

Common beginner issue

If `docker info` says "Cannot connect to the Docker daemon", start Docker Desktop / Engine and retry.

Checkpoint

Before continuing: you can run `docker info` without an error.

Lab project files

Folder layout

```
localloop-api/
  src/app.js
  src/server.js
  test/app.test.js
  package.json
  package-lock.json
  Dockerfile
  .dockerignore
  compose.yaml
```

Why so few files?

- Uses Node's built-in HTTP API
- No extra npm dependencies
- Built-in `node:test` tests
- Focus is Docker lifecycle

Lab 1.1: run the Node.js project

```
cd localloop-api
npm ci
npm test
npm start
```

- `npm ci`: installs precisely from the included lockfile (no extra packages needed here).
- `npm test`: runs three automated API tests.
- `npm start`: starts the API on port 3000.

Checkpoint

Terminal prints “LocalLoop API listening on port 3000”. Keep this terminal open.

Lab 1.2: use the browser

Open these addresses:

1. `http://localhost:3000/` — application name/version.
2. `http://localhost:3000/health` — `{"status":"ok"}`.
3. `http://localhost:3000/api/listings` — existing food listing.
4. `http://localhost:3000/api/stats` — listing counts.

Remember

A browser address bar is useful for **GET**. To send **POST**, use a terminal or API client.

Checkpoint

Your local API responds before you start Docker.

LocalLoop API: the request map

Method	Path	Purpose
GET	`/health`	Read health status
GET	`/api/listings`	View available data
GET	`/api/stats`	View summary counts
POST	`/api/listings`	Create a listing
POST	`/api/listings/:id/reserve`	Reserve a listing

HTTP responses: 200 (OK), 201 (created), 400 (bad data), 404 (missing), 409 (conflict). Learning simplification:

accounts, permissions, database persistence and safety approval are deliberately not implemented.

Lab 1.3: create a food listing

```
curl -X POST http://localhost:3000/api/listings \
  -H "Content-Type: application/json" \
  -d '{"title":"Fruit box","quantity":4,
       "pickupLocation":"Campus library"}'
```

macOS/Linux / Bash example above. For Windows PowerShell, see the next slide.

Checkpoint

Expected: HTTP 201 and a new JSON object with `status:"available"`.

Windows PowerShell: POST request

```
$body = @{ title = "Fruit box"; quantity = 4;
  pickupLocation = "Campus library" } | ConvertTo-Json
Invoke-RestMethod -Uri http://localhost:3000/api/listings `
  -Method Post -ContentType "application/json" -Body $body
```

Two shell syntaxes

Use `curl` in Bash, or `Invoke-RestMethod` in PowerShell. Do not mix the line-continuation characters (\ vs. backtick).

Lab 1.4: reserve and validate

```
# Reserve the listing with ID 1
curl -X POST http://localhost:3000/api/listings/1/reserve

# Run it again: the same reservation must be rejected
curl -X POST http://localhost:3000/api/listings/1/reserve

# Recheck counts
curl http://localhost:3000/api/stats
```

Expected: First reservation 200; second returns a JSON conflict (HTTP 409).

Checkpoint

Explain why returning 409 is better than silently reserving the same food twice.

What makes the demo event-driven?

```
const events = new EventEmitter();
events.on('listing.created', item => {
  console.log('Created', item.id);
});

// When an item is created:
events.emit('listing.created', item);
```

- A **producer** emits a domain event.
- A **listener** responds without changing the HTTP route's purpose.
- Here, events only write logs; they are *not* durable messages.

For the full LocalLoop platform, notifications or audit processing could become separate services.

Knowledge check: before containerising

1. What must be running for `localhost:3000` to respond?
2. Is `GET /api/listings` a read or a write operation?
3. Why is the data lost when we restart the teaching API?
4. What does `npm test` verify in this project?

Checkpoint

Stop the locally running server using `Ctrl+C` to free port 3000 before the Docker lab.

Dockerfile: our packaging recipe

```
FROM node:24-alpine
WORKDIR /app
ENV NODE_ENV=production
COPY package*.json ./
RUN npm ci --omit=dev
COPY --chown=node:node src ./src
USER node
EXPOSE 3000
CMD ["node", "src/server.js"]
```

Build-time: `FROM`, `COPY`, `RUN`. **Run-time:** `USER`, `CMD`; `EXPOSE` documents an intended port.

Read a Dockerfile like a recipe

Instruction	Meaning
`FROM`	Start from a Node.js base image
`WORKDIR`	Set working directory inside image
`COPY`	Copy project files into image
`RUN`	Execute command *while building* (install packages)
`ENV`	Set a runtime environment variable
`USER`	Run the application as non-root `node` user
`EXPOSE`	Document application port; *does not* publish it
`CMD`	Default command when container starts

Official reference: https://docs.docker.com/reference/dockerfile/

Why we need .dockerignore

```
**/node_modules
.git
.env
.env.*
**/.npmrc
coverage
*.log
test
```

- Makes build context smaller, cleaner and faster.
- Helps avoid copying local dependencies, secrets, history and tests into runtime images.
- This does **not** replace proper secret management or code review.

Lab 2.1: build your image

```
docker build -t localloop-api:1.0 .
docker images
```

Read the command:

- `docker build`: build from the current directory's Dockerfile.
- `-t localloop-api:1.0`: assign a name and version tag.
- `.`: build context = current folder.

Checkpoint

The image named `localloop-api:1.0` appears in `docker images`.

Build layers and caching

1. Base image layer (`node:24-alpine`).
2. Project manifests copied (`package.json`, lockfile).
3. Dependencies prepared (`npm ci`).
4. Application source copied.

Small change, faster rebuild

If only `src/app.js` changes, Docker can often reuse earlier dependency layers. The order of `COPY` steps matters.

Checkpoint

Change a response string, rebuild as `1.1` and observe which steps are cached.

Lab 2.2: run a container

```
docker run -d --name localloop \
  -p 3000:3000 localloop-api:1.0

docker ps
docker logs localloop
```

- `-d`: detached/background mode.
- `-name`: convenient container identifier.
- `-p HOST:CONTAINER`: publish port to host.

Checkpoint

Browse `http://localhost:3000/health`: you should get the same JSON as before.

Port mapping visualised



Important difference

`EXPOSE 3000` in a Dockerfile is *documentation*; `-p 3000:3000` publishes the port.

Lab 2.3: inspect the running container

```
docker ps
docker logs localloop
docker exec localloop node --version
docker inspect localloop
```

- `ps`: which containers are running?
- `logs`: what did the application print?
- `exec`: run a process in a running container.
- `inspect`: detailed JSON configuration and metadata.

Checkpoint

Find the image tag and port mapping for your container.

Lab 2.4: stop and remove

```
docker stop localloop
docker ps -a
docker rm localloop
```

What remains?

The **image** remains in `docker images`; the named **container** has been removed.

Try again: run a new container from `localloop-api:1.0`. It begins with the original in-memory listing, not your earlier changes.

Container state is usually ephemeral

In our teaching API

- Listings stored in a JavaScript array
- New process starts with fresh array
- Another container has a different copy

In a real LocalLoop system

- Store records in a database
- Use volumes for stateful services
- Define backups and recovery

Rule: never rely on a container's writable layer for durable business data.

Troubleshooting: 5 common problems

Symptom	First check / remedy
Cannot connect to Docker daemon	Start Docker Desktop or Engine; `docker info`
Port 3000 already allocated	Stop local Node server/container or use `-p 3001:3000`
Cannot find Dockerfile	Use project root: `cd localloop-api`
Cannot reach application	Run `docker ps`, `docker logs`; check `-p`
Image build fails at npm ci	Verify matching `package.json` and `package-lock.json`

Checkpoint

Debug in layers: engine → image → container → port → application.

Lab 3: why Compose?

Instead of memorising a long `docker run` command, describe the service once in `compose.yaml`.

Compose is useful when

- A project has several services (API + database + worker).
- A team needs repeatable local setup.
- Configuration, ports, environment and dependencies should be versioned.

Today: one API service, as a stepping stone to multi-container systems.

Compose configuration

```
services:
  api:
    build: .
    image: localloop-api:1.0
    ports:
      - "3000:3000"
    environment:
      PORT: "3000"
      NODE_ENV: production
    restart: unless-stopped
```

Interpretation: build the API image, publish port 3000, and set configuration. Use modern `docker compose` (with a space), not legacy `docker-compose`.

Lab 3.1: run with Compose

```
docker compose config
docker compose up --build -d
docker compose ps
docker compose logs api

# When finished:
docker compose down
```

- `config`: validate and show the resolved configuration.
- `up -build -d`: build and start the service.
- `down`: remove Compose-created containers/network.

Checkpoint

Start with Compose and verify `http://localhost:3000/health`.

Environment configuration

Good practice

- Use `PORT` and `NODE_ENV` as configuration
- Keep dev and production settings separate
- Prefer explicit ports and names

Avoid

- Passwords in Dockerfiles
- API keys in images / Git
- Logging sensitive information

For real deployment, use a secrets manager or Docker/Compose secrets where appropriate; `ENV` is *not* a secret vault.

Test and review before distribution

1. **Source quality:** code review and static inspection.
2. **Functional tests:** run `npm test` and verify API behaviour.
3. **Container smoke test:** start image and test `/health`.
4. **Runtime inspection:** check logs, user, port and configuration.
5. **Security review:** exclude secrets, validate inputs, use least privilege.

COMP3028 connection

For code reviews, discuss a Dockerfile or Compose PR as seriously as application code: correctness, security, maintainability and test evidence.

Security basics for a Node image

- Use a **supported LTS** Node base image; update and rebuild regularly.
- Prefer non-root `USER node`; restrict exposed network interfaces.
- Exclude credentials, `.env` and private files from images.
- Use a lockfile and `npm ci`; review dependencies when added.
- Apply API authentication, authorisation, input validation and safe error handling.
- Consider image scanning and fixed immutable digests in real deployments.

Docker best practices: https://docs.docker.com/build/building/best-practices/

Images need names and versions

`yourname/localloop-api:1.0`

Part	Example	Role
Namespace	`yourname`	Docker Hub account / organisation
Repository	`localloop-api`	Application image name
Tag	`1.0`	Version label

Versioning habit

Prefer explicit tags such as `1.0` or a commit-derived tag over relying on `latest`.

Lab 4.1: publish to Docker Hub (optional)

```
# Replace YOUR_ID with your own Docker Hub username
docker login
docker tag localloop-api:1.0 YOUR_ID/localloop-api:1.0
docker push YOUR_ID/localloop-api:1.0
```

- You need a Docker Hub account and permission to push the repository.
- Keep images free of credentials and private project data.
- If campus network blocks Hub, use the offline method next.

Checkpoint

Your account's repository page shows a `1.0` image tag (if you opt in).

Lab 4.2: pull and run the shared image

```
docker pull YOUR_ID/localloop-api:1.0
docker run --rm -p 3001:3000 \
  YOUR_ID/localloop-api:1.0
```

Now open: `http://localhost:3001/health`

Why 3001?

The host port can differ from the port inside the container. This also avoids conflict if port 3000 is already used.

In class, you can pull a teammate's image if it is publicly accessible or you have appropriate access.

Alternative: distribute an image offline

```
# On sending computer
docker save -o localloop-api-1.0.tar localloop-api:1.0

# Transfer the .tar through an approved channel
# On receiving computer
docker load -i localloop-api-1.0.tar
docker run --rm -p 3001:3000 localloop-api:1.0
```

Trade-off: useful without a registry, but large files and manual version tracking. Be careful with large tar files and never package secrets.

Live exercise: Docker trouble ticket

Scenario

A student runs `docker run -p 3000:3000 localloop-api:1.0`, but gets "port is already allocated".

Task (3 min):

1. Identify at least two plausible processes holding the host port.
2. Find a safe way to run the container without stopping other services.
3. State the new browser URL.

Checkpoint

Answer: publish `-p 3001:3000`; browse `localhost:3001`. Also inspect running containers and local Node processes.

Extension tasks: choose one

1. **Basic:** Add `GET /api/listings/available` and a matching test. Rebuild as `1.1`.
2. **Intermediate:** Add a validated category field; include it in creation and response.
3. **Intermediate:** Add a health-check instruction or Compose service healthcheck.
4. **Advanced:** Introduce a database service and persistent storage; justify data lifecycle.

Evidence: source change, passing test, image tag, and output from the running container.

Week 11 vs. assessment requirements

What this lab is

A supported practice activity connecting application development, testing, containerisation, service configuration and distribution.

What it is not

A new official Assessment 2 marking rule. Continue to follow the official code-review brief, vUWS submission instructions and Learning Guide rubric.

Useful PR discussion: explain why Dockerfile changes are safe, what commands prove they work, and what issues remain.

Student demonstration checklist

1. `npm test` passes.
2. `docker build -t localloop-api:1.0 .` succeeds.
3. `docker run -p 3000:3000 ...` starts an accessible service.
4. `/health`, `/api/listings` and reserve flow respond correctly.
5. `docker ps` and `docker logs` show evidence.
6. `docker compose up -build -d` works.
7. Can explain `Dockerfile`, `.dockerignore`, image tag and port map.

Checkpoint

Show understanding rather than only screenshots or copied commands.

Quick knowledge check

1. What is the difference between an **image** and a **container**?
2. Why copy `package*.json` before `src/`?
3. Why does `EXPOSE 3000` not make the API publicly reachable?
4. How is `docker compose up` different from `docker build`?
5. How would you deliver the `1.0` image to another computer?

Exit ticket: Write one Docker command and explain it in your own words.

Recap: 5 commands to remember

```
npm test
docker build -t localloop-api:1.0 .
docker run --rm -p 3000:3000 localloop-api:1.0
docker compose up --build -d
docker tag localloop-api:1.0 YOUR_ID/localloop-api:1.0
```

Source $\rightarrow$ Test $\rightarrow$ Image $\rightarrow$ Container $\rightarrow$ Distribution

Official reading and documentation

- **Docker Docs.** Node.js guide: `https://docs.docker.com/guides/nodejs/`
- **Docker Docs.** Dockerfile reference: `https://docs.docker.com/reference/dockerfile/`
- **Docker Docs.** Compose quickstart: `https://docs.docker.com/compose/gettingstarted/`
- **Docker Docs.** Image sharing: `https://docs.docker.com/docker-hub/repos/manage/hub-images/push/`
- **Node.js.** Release schedule: `https://nodejs.org/en/about/previous-releases`
- **Unit documents.** COMP3028-SPR26 Case Study and Assessment 2 Code Review brief.

Note: Technical instructions checked against official documentation in November 2026.

Build once. Run consistently. Share deliberately.

Next: containerise a feature of your team's LocalLoop application.

Discuss: Which services deserve their own containers? Which data must persist?